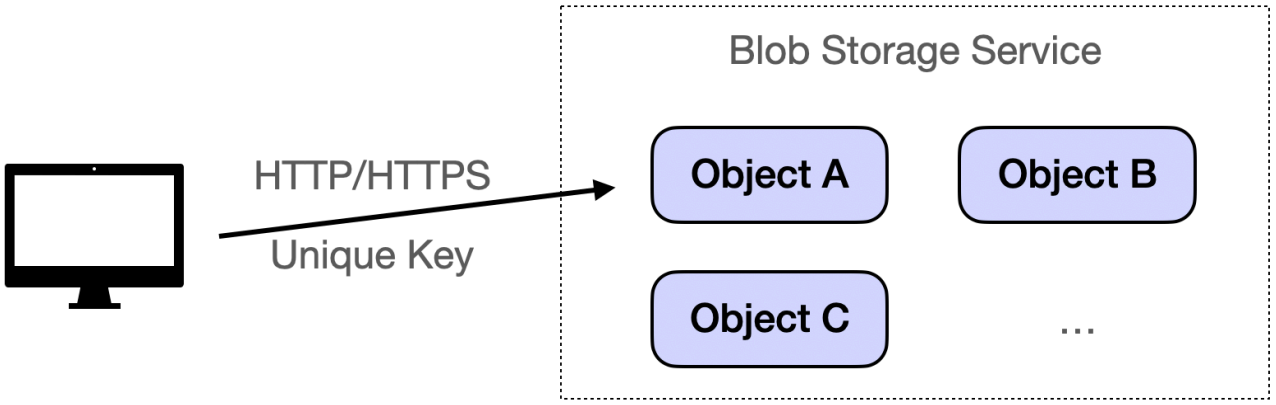
Revolutionizing Data Management with Scalable, Durable, and Cost-Effective Solutions

systemdesignschool.io/fundamentals/blob-object-storage



Blob/Object Storage

Full-text search databases handle text within documents. But applications also store binary files: images, videos, PDFs, backups. These files don't fit in traditional databases.

The File System Problem

A video streaming service stores millions of videos. Each video ranges from megabytes to gigabytes. Traditional file systems organize files in directories: `/videos/2024/01/video1.mp4`.

This hierarchy creates problems at scale. Finding a specific video requires traversing directories. List all videos uploaded in January? Scan thousands of subdirectories. The file system wasn't designed for billions of files.

Databases can store binary data as BLOBs. But inserting a 1GB video into PostgreSQL is impractical. The database loads the entire blob into memory. Backups become enormous. Replication slows down. Databases optimize for structured data, not large binary files.

Cloud applications need to serve files globally. A user in Tokyo requests a video. The file lives in a US datacenter. Latency suffers. File systems lack built-in global distribution.

How Object Storage Works

Object storage treats each file as an independent object. Upload a video. The system assigns a unique ID and stores it in a flat namespace. No directories. No hierarchy.

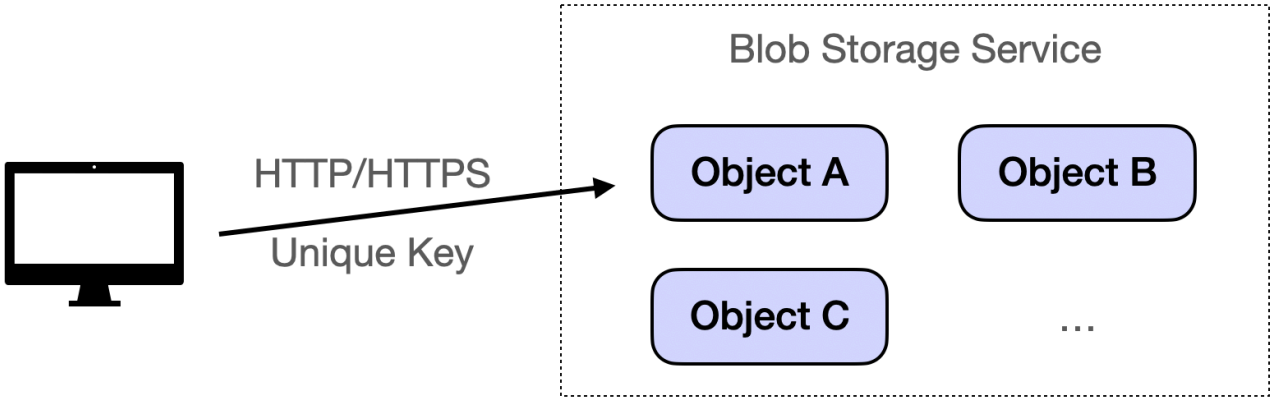
Each object contains three parts: the data itself, metadata, and a unique key. The key might be `videos/paris.mp4`. Metadata stores content type, upload date, and custom tags. Access the object by key over HTTP.

```
{ "ObjectKey": "videos/paris.mp4", "Metadata": { "ContentType": "video/mp4", "UploadDate": "2024-01-15", "Duration": "120s" }, "Data": "Binary video data..." }
```

Objects are stored in buckets. A bucket is a top-level container. Create a bucket called `my-videos`. Store millions of objects inside. Each object has a unique key within the bucket.

HTTP access enables global distribution. Request `https://storage.example.com/my-videos/paris.mp4`. The storage system returns the file. CDNs can cache objects at edge locations. Users get low latency worldwide.

Flat namespace scales infinitely. No directory traversal needed. Look up objects by key using hash tables. Performance stays constant whether you have 1,000 objects or 1 billion.



Built-in Durability

Object storage replicates data automatically. Upload an object to AWS S3. S3 copies it to multiple servers in different availability zones. One server fails? The system serves the file from another copy. Data durability reaches 11 nines: 99.999999999%.

Versioning protects against accidental deletion. Delete an object. The system marks it as deleted but keeps the data. Retrieve previous versions anytime. This prevents data loss from application bugs or user errors.

When to Use Object Storage

Multimedia content fits naturally. YouTube stores billions of videos. Netflix serves petabytes of streaming content. Each video becomes an object. HTTP URLs enable direct browser access. No application server needed.

Backup and disaster recovery benefit from durability and scale. Database backups grow to terabytes. Object storage handles this scale cheaply. Geographic replication protects against datacenter failures.

Static website hosting uses object storage directly. Upload HTML, CSS, JavaScript, and images. Configure the bucket for static hosting. The storage system serves files over HTTP. This costs less than running web servers.

Data lakes collect raw data for analytics. IoT sensors generate millions of events per day. Store raw JSON or CSV files as objects. Analytics tools like Spark read directly from object storage. This separates storage from compute.

Popular Services

Amazon S3 leads the market. It provides versioning, lifecycle policies, and cross-region replication. Most AWS services integrate with S3. Lambda functions can trigger on S3 uploads. Analytics tools read S3 data directly.

Google Cloud Storage offers similar features with tighter integration to Google's analytics stack. BigQuery queries data in Cloud Storage directly. No ETL needed.

Azure Blob Storage provides tiered storage: hot for frequently accessed data, cool for backups, archive for long-term storage. Each tier has different pricing and access speeds.

Using object storage is straightforward:

```
import boto3 s3 = boto3.client('s3') # Upload a file s3.upload_file('video.mp4', 'my-bucket', 'videos/video.mp4') # Download a file s3.download_file('my-bucket', 'videos/video.mp4', 'local-video.mp4') # Delete a file s3.delete_object(Bucket='my-bucket', Key='videos/video.mp4')
```